

# Introduction to Python

Week 4

# Program for This Week

1. Defining strings
2. Basic string manipulation
3. String methods
4. F-strings
5. Object-oriented programming (OOP) concepts
6. Error handling
7. Midterm and homework questions (next week)

# Defining strings with quotes

- When using single quotes, Python searches for a matching quote, and stops searching at the *End of the Line* (EOL)
  - The following give an error (SyntaxError: EOL while scanning string):
    - `print("it')`
    - `print('it")`
- When using triple quotes, Python searches for a matching triple, and stops searching at the *End of the File* (EOF)
  - The following give an error (EOF while scanning string literal):
    - `print(''''it's  
a new line)`
    - `print("""it's  
another new line  
""")`

# Quotes inside a string

- The following works:
  - `print("it's")` # Onscreen: it's
  - `print('it"s')` # Onscreen: it"s
- The following gives an error (`SyntaxError: invalid syntax`):
  - `print("it"s")`
  - `print('it's')`
- Alternatively, working with *escape characters* also works:
  - `print("it\"s\'s")` # Onscreen: it"s's
  - `print('it\"s\'s')` # Onscreen: it"s's
- Triple quotes work as well:
  - `print("""it"s's""")` # Onscreen: it"s's
  - `print(''''it"s's''')` # Onscreen: it"s's

# Escape characters

- `\\` = `\`
- `\'` = `'`
- `\"` = `"`
- `\n` = go to the next line
- `\t` = insert a tab
- `print('The info, it\'s on the next\\following line in two columns:\\n1\\t2')`  
# Onscreen: The info, it's on the next\\following line in two columns:  
# 1 2
- This is great, but if you must type the string yourself, using triple single or double quotes may be easier:
- `print("""The info, it's on the next\\following line in two columns:  
1 2""")`  
# Onscreen: The info, it's on the next line in two columns:  
# 1 2

# Basic string manipulation: built-in functions

- There are also some built-in functions that can be used with strings
  - *Built-in* means that they are part of the Python system, you don't have to program them yourself
- The function `len()` gives you the length of a string:
  - ```
a = 'abcd'
print(len(a)) # Onscreen: 4
```
- *Concatenation*, which is used to concatenate strings, uses the `+` operator:
  - ```
a = 'abcd'
b = 'ef'
a = a + b
print(a) # Onscreen: abcdef
```
- That you can use an operator for several different operations is called *operator overloading*, in this case the `+` operator to add numbers, to concatenate strings, to append to lists
- The function `sorted()` can also be used to a string:
  - ```
print(sorted('acbd')) # Onscreen: ['a', 'b', 'c', 'd']
```
  - The result of sorting a string is a list. Later this lecture we will use the `join` method to transform this list back into a string

# String methods

- Methods are functions
- Methods are, just like functions, called by adding round brackets after the method
  - Within these round brackets you can add arguments that you want to pass to the method
- The difference with a function is that you always must pass the object on which the method is applied as first argument to the method
  - This can be done in several ways, but most easily by using the dot operator (dot = .)
- Compare a function call:
  - `print(len('abcd'))` # Onscreen: 4
- With a method call:
  - `'abcd'.index('b')` # Onscreen: 1
- Though it seems like only one argument is passed to the method, two arguments are passed to this index method: the string object with value 'abcd', and an object with value 'b'

# String methods

- Methods can either:
  - Change the object that is passed to them with the dot operator
  - Return a changed version of the object as a new object
- If you program methods yourself, you can even let them do both
- You can assume that the built-in methods of the standard library take only one option
  - For a built-in method, you must know which option it will be
  - With methods used in the Pandas library you can often change the default option via a parameter
- Question: What will the option be in the case of a string method?
  - As strings are immutable, string methods can only return a new object
- Technically, even a method that changes an object returns a value: just like a function without a return statement, a method without a return statement will return `None`
  - However, that returned value is not what you will be interested in



# String methods: Question

- For example, if you want to capitalize (make the first character uppercase) a string, which of the following code fragments will print: `Abcd`

```
1. s1 = 'abcd'  
   s1 = s1.capitalize()  
   print(s1)
```

```
2. s1 = 'abcd'  
   s1.capitalize()  
   print(s1)
```

- a) Both are correct
- b) Both are incorrect
- c) Only 1 is correct
- d) Only 2 is correct

# String methods: Question

- For example, if you want to capitalize (make the first character uppercase) a string, which of the following code fragments will print: `Abcd`

```
1. s1 = 'abcd'  
   s1 = s1.capitalize()  
   print(s1)
```

```
2. s1 = 'abcd'  
   s1.capitalize()  
   print(s1)
```

- a) Both are correct
  - b) Both are incorrect
  - c) Only 1 is correct
  - d) Only 2 is correct
- Correct answer: c. `s1.capitalize` doesn't change string `s1`, but returns a new string. You must give that new string a name else you cannot use it in the print statement. The second code fragment, actually prints `abcd`

# String methods: change and test case

- Change the case:
  - `print('ABcd'.upper())` # Onscreen: ABCD
  - `print('ABcd'.lower())` # Onscreen: abcd
- Change only the first character into uppercase:
  - `print('abcd'.capitalize())` # Onscreen: Abcd
- NB! I use the word change a lot, but don't forget the string cannot be changed, we actually create a new string on the basis of the old string
- Test the case of the letters in the string:
  - `print('ABCD'.isupper())` # Onscreen: True
  - `print('ABcd'.isupper())` # Onscreen: False
  - `print('ABcd'.islower())` # Onscreen: False
  - `print('abcd'.islower())` # Onscreen: True
  - `print('ABcd'[:2].isupper())` # Onscreen: True

# String methods: split

- Split a string: for example, consider `s1 = 'axbcxdefxg h\nij\tk'`
  - `print(s1.split('x'))` # Onscreen: `['a', 'bc', 'def', 'g h\nij\tk']`
  - `print(s1.split('x', 2))` # Onscreen: `['a', 'bc', 'defxg h\nij\tk']`
  - `print(s1.rsplit('x', 2))` # Onscreen: `['axbc', 'def', 'g h\nij\tk']`
  - `print(s1.split())` # Onscreen: `['axbcxdefxg', 'h', 'ij', 'k']`
  - `print(s1.split('q'))` # Onscreen: `['axbcxdefxg h\nij\tk']`
- The argument you pass to `split`, is used to split the string and is also removed from the string
  - If the argument is not in the string, Python will return an element with only one element, which is a copy of the string
  - If you also add a number, then that will be the maximum number of times Python will split the string
- The default is splitting around spaces, new lines (`\n`), and tabs (`\t`)
- To split a string in physical lines:
  - `s1 = 'axbcx\rdefxg \nhiij '`
  - `print(s1.split('\n'))` # Onscreen: `['axbcx\rdefxg ', 'hiij']`
  - `print(s1.splitlines())` # Onscreen: `['axbcx', 'defxg ', 'hiij']`
- I prefer the first

# String methods: join

- Join a string: for example: `l1 = ['abc', 'de', 'fg', 'h']`
  - `print('xy'.join(l1))` # Onscreen: `abcxydxyfgxyh`
  - `print(''.join(l1))` # Onscreen: `abcdefgh`
- How to sort a string:
  - `a = sorted('0123456789', key=lambda x: str(int(x) % 2 == 1) + x)`
  - `print(''.join(a))` # Onscreen: `'0246813579'`

# String methods: strip

- Strip a string: for example, consider `s1 = ' fgedee '`
  - `print(s1.strip(' e'))` # Onscreen: `fged`
  - `print(s1.strip())` # Onscreen: `fgedee`
  - `print(s1.lstrip(' e'))` # Onscreen: `fgedee..`
  - `print(s1.lstrip())` # Onscreen: `fgedee..`
  - `print(s1.rstrip(' e'))` # Onscreen: `..fged`
  - `print(s1.rstrip())` # Onscreen: `..fgedee`
- Python removes characters from the object if they are in the passed string and stops as soon as it encounters a character that is not in the list
- `strip` works from both sides of the string, `lstrip` starts from the left side, and `rstrip` starts from the right side
  - `s1.strip(' e')` and `s1.lstrip(' e').rstrip(' e')` give the same result
- The default is removing spaces, new lines (`\n`), and tabs (`\t`)
  - More types of so-called white spaces will be removed, but only these 3 you must know
- NB! The spaces in the strip examples are represented by dots in the Onscreen

# String methods: replace

- Replace a substring: consider for example `s1 = 'abcdefghabcdefgh'`:
  - `print(s1.replace('a', 'aa'))` # Onscreen: `aabcdefghaabcdefgh`
  - `print(s1.replace('a', 'aa', 1))` # Onscreen: `aabcdefghabcdefgh`
  - `print(s1.replace('a', ''))` # Onscreen: `bcdefghbcdefgh`
  - `print(s1.replace('q', 'qq'))` # Onscreen: `abcdefghabcdefgh`
- The first passed string is the old substring and it will be replaced by the second passed string, that is the new substring
- The new and the old substring can have any length
  - Only the new string can be an empty string: you can use this to remove substrings from the string
- You can add an integer, and that will tell Python how many changes to make
  - If you don't add a number Python will replace all old substrings

# String methods: find, index

- Find a substring: consider for example
  - `s1 = 'abcdefghabcdefgh'`
  - `print(s1.find('e'))` # On screen: 4
  - `print(s1.index('e', 5))` # On screen: 12
  - `print(s1.find('e', 5, 10))` # On screen: -1
  - `print(s1.index('e', 5, 13))` # On screen: 12
  - `print(s1.find('ef'))` # On screen: 4
  - `print(s1.index('q'))` # On screen: ValueError: substring not found
- With these methods, you can find the index of a substring in a string
- If you add one integer as an extra parameter, it will tell Python from where to start searching in the string
- If you add another integer as an extra parameter, it will tell Python before which index to stop searching
- The only difference between `find` and `index` is that if `find` cannot find anything, `find` will return -1 and if `index` cannot find anything `index` will throw an error. In all other cases `find` and `index` will return a non-negative integer.



# String methods: count

- Find a substring: consider for example
  - `s1 = 'abcdefghabcdefgh'`
  - `print(s1.count('e'))` # Onscreen: 2
  - `print(s1.count('e', 5))` # Onscreen: 1
  - `print(s1.count('e', 5, 10))` # Onscreen: 0
  - `print(s1.count('e', 5, 13))` # Onscreen: 1
  - `print(s1.count('ef'))` # Onscreen: 2
  - `print(s1.count('q'))` # Onscreen: 0
- With these methods, you can find how often a substring occurs in a string
- If you add one integer as an extra parameter, it will tell Python from where to start searching
- If you add another integer as an extra parameter, it will tell Python before which index to stop searching

# String methods: Question

- You have a string called 's1', with only lowercase letters. You want to create a new string where only the last letter is capitalized and give it the name 's1'. Which of the following codes, gives the correct result?

1. `s1 = s1[::-1].capitalize()[::-1]`

2. `s1 = s1[:-1] + s1[-1].upper()`

3. `s1 = ''.join(list(s1)[:-1] + [list(s1)[-1].upper()])`

- a) Only 1 is correct
- b) Only 2 is correct
- c) They are all correct
- d) They are all incorrect

# String methods: Question

- You have a string called 's1', with only lowercase letters. You want to create a new string where only the last letter is capitalized and give it the name 's1'. Which of the following codes, gives the correct result?

1. `s1 = s1[::-1].capitalize()[::-1]`

2. `s1 = s1[:-1] + s1[-1].upper()`

3. `s1 = ''.join(list(s1)[:-1] + [list(s1)[-1].upper()])`

a) Only 1 is correct

b) Only 2 is correct

c) They are all correct

d) They are all incorrect

e) Correct answer: c. This is quite a difficult question best to run it and see what happens, and try to understand

# String methods: Question

- How to sort a string?

```
1. s1 = 'acbd'  
   l1 = sorted(s1)  
   s1 = ''.join(l1)
```

```
2. s1 = 'acbd'  
   l1 = list(s1)  
   l1 = l1.sort()  
   s1 = ''.join(l1)
```

- a) Both are correct
- b) Both are incorrect
- c) Only 1 is correct
- d) Only 2 is correct

# String methods: Question

- How to sort a string?

```
1. s1 = 'acbd'  
   l1 = sorted(s1)  
   s1 = ''.join(l1)
```

```
2. s1 = 'acbd'  
   l1 = list(s1)  
   l1 = l1.sort()  
   s1 = ''.join(l1)
```

- a) Both are correct
- b) Both are incorrect
- c) Only 1 is correct
- d) Only 2 is correct

- Correct answer: c. In the third line of code in option 2, `l1.sort()`, changes `l1` and returns `None`, and so in line 4 `l1` is now `None` and Python will throw an error

# String module

- ```
import string
print(string.ascii_lowercase) # Onscreen: abcdefghijklmnopqrstuvwxyz
print(string.ascii_uppercase) # Onscreen: ABCDEFGHIJKLMNOPQRSTUVWXYZ
print(string.ascii_letters)
# Onscreen: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
print(string.digits) # Onscreen: 0123456789
print(string.punctuation) # Onscreen: "#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
~
print(string.whitespace) # Onscreen:
print ('\\t' in string.whitespace, '\\n' in string.whitespace, ' '
in string.whitespace) # Onscreen: True True True
```
- These are all strings that you can use after importing string
- In general, laziness is a good thing for a programmer. Although it is hardly more work, using these constants instead of typing all those characters yourself, avoids making errors.

# F-strings

- *F-strings* can be used to format output
  - They can be very convenient
- Say you want a function that multiplies two integers and e.g., when you multiply 7 and 13, returns a string that looks as follows: "0007 times 0013 gives 0091"
- This can be done by writing a function with what we learned in the first 3 weeks:

```
def main(a, b):  
    def format1(x):  
        return ('0000' + str(x))[-4:]  
    return format1(a) + ' times ' + format1(b) + ' gives ' + format1(a * b)
```

# F-strings

- With an F-string, the task gets more convenient and more readable:
  - ```
def main(a, b):  
    return f'{a:04d} times {b:04d} gives {a * b:04d}'
```
- Inside the curly braces, before the colon you have the value and after the colon the formatting of that value
  - You don't have to know the part after the colon for the exam, just the value part
- In this case, the `f` means you deal with an integer, the 4 gives you the length, and the 0 tells Python to add extra zeroes in front if the number is shorter than 4 digits



# F-strings

- If you leave out the formatting part, Python will just apply the `str` function to the value
  - `print(f'{a} {b}' == str(a) + ' ' + str(b))` # Onscreen: True
- The values can be just simple values, or the returns of functions/methods, or slices or values from sequences or dictionaries: in short, all expressions that Python will evaluate to a value

- For example:

- ```
s1 = 'tekst'
t1 = (1, 2, 3)
l1 = [4, 5, 6]
capitals = {'Netherlands' : 'Amsterdam'}
print(f'{s1} {s1.upper()} {t1[2]} {l1} {capitals["Netherlands"]}')
# Onscreen: tekst TEKST 3 [4, 5, 6] Amsterdam
```

# F-strings

- Just like with strings in general, if you use one type of quotes on the outside, you should use the other type of quotes on the inside
  - If you cannot solve this problem, you can always use escape characters (\' and \")
- If you need to escape a { or } you cannot achieve that with a backward slash, but you can by doubling the braces {{ and }}
- Although f-strings are very convenient, I find the documentation on the internet a bit difficult to comprehend, so I always end with trial and error, and success in the end
- A nice trick for debugging purposes:

```
a = 10
b = 'Amsterdam'
print (f'{a={}}{b=}')      # Onscreen: a=10 b='Amsterdam'
```

# Object-Oriented Programming

- Programming philosophy/approach
  - A way of organising code
- Well-organised code: easier to understand, fix bugs and maintain/extend
- Trade-off with performance
- Usually only relevant in large projects
- OOP is helpful for organisation, but not a guarantee for good organisation
- (Alan Kay, “inventor” of OOP: better named “process-oriented programming”)

# Object-Oriented Programming (Background)

- OOP is hard-wired into Python, but we can do more of it.
- Everything is an object/process
- Encapsulation: objects have data/attributes (name, age, colour) and methods/stuff we want to do with the data (index, sort, draw)
- Abstraction: focus on what object does, hide how it does it.
- Inheritance: objects take properties (attributes and methods) from superclasses (circle is a type of shape, consumer is a type of agent)
- Polymorphism: conceptually similar methods have same name, even if implementation is different (draw circle vs draw rectangle)

# Object-Oriented Programming (Background)

- Advantage: if well chosen abstractions, focus on problem at hand
- Easier to read, fix and extend
  - Compare `list1.len()` and

```
length = 0
for x in list1:
    length +=1
return length
```
- If not well-chosen, classes can be a confusing mess
- Choosing well is beyond the scope of this course

# Class definition

```
class Rectangle:
    def __init__(self, length, width=1):
        self.length = length
        self.width = width
    def size(self, data_type=int):
        return f"The size = {data_type(self.length * self.width)}."

rectangle_1 = Rectangle(2,3)
rectangle_2 = Rectangle(3)
rectangle_3 = Rectangle(2,2)
for object in [rectangle_1, rectangle_2, rectangle_3, Rectangle(4,4)]:
    print(object.size(float))

# On screen: 6.0
             3.0
             4.0
             16.0
```

# Class definition

- A class definition starts with the `class` keyword
- The first line of the class definition ends with a colon (:), so the next line(s) should be indented
- Everything indented more than the `class` keyword is seen as part of the class definition
  - The class definition stops when the level of indentation is the same as that of the `class` keyword

# Object creation

```
class Rectangle:
    def __init__(self, length, width=1):
        self.length = length
        self.width = width
    def size(self, data_type=int):
        return f"The size = {data_type(self.length * self.width)}."

rectangle_1 = Rectangle(2,3)
rectangle_2 = Rectangle(3)
rectangle_3 = Rectangle(2,2)

for object in [rectangle_1, rectangle_2, rectangle_3, Rectangle(4,4)]:
    print(object.size())
```



# Object creation

- `rectangle_1 = Rectangle(2, 3)`
- Python will:
  1. Create a new object (empty at this point)
  2. Pass that object and the arguments between the brackets, in this case 2 and 3, to the `__init__` method that is defined inside the class definition
    - Python calls the `__init__` method in the following way: `__init__(rectangle_1, 2, 3)`
  3. Assign the name `rectangle_1` to the new object
- The creation of an object this way, is called the *instantiation* of the object

# Class name

```
class Rectangle:
    def __init__(self, length, width=1):
        self.length = length
        self.width = width
    def size(self, data_type=int):
        return f"The size = {data_type(self.length * self.width)}."

rectangle_1 = Rectangle(2,3)
rectangle_2 = Rectangle(3)
rectangle_3 = Rectangle(2,2)

for object in [rectangle_1, rectangle_2, rectangle_3, Rectangle(4,4)]:
    print(object.size())
```

- It is a convention to start class names with a capital letter
  - Convention means: Python doesn't care, but people do

# Methods

```
class Rectangle:
```

```
    def __init__(self, length, width=1):  
        self.length = length  
        self.width = width
```

```
    def size(self):  
        return f"The size = {self.length * self.width}."
```

```
rectangle_1 = Rectangle(2,3)
```

```
rectangle_2 = Rectangle(3)
```

```
rectangle_3 = Rectangle(2,2)
```

```
for object in [rectangle_1, rectangle_2, rectangle_3, Rectangle(4,4)]:  
    print(object.size())
```

# Methods

- Definitions of functions and methods look the same
- The difference is that a method always expects an object as the first argument
- The object itself is passed to the method via the dot operator:
  - The method call `rectangle_1.size(float)` passes the object with the name `rectangle_1` to the method as the first argument, `float` is the second argument
- By convention, the corresponding parameter is called `self`
- You call these methods with the help of round brackets, pass the object via the dot operator, and pass other arguments between the brackets
  - `print(rectangle_1.size())` # Onscreen: 6
  - `print(rectangle_2.size())` # Onscreen: This rectangle is 3 by 1
- You have two type of methods:
  - Methods you call directly like `size()`
  - Methods that you call indirectly like `__init__`

# `__init__` method

```
class Rectangle:
```

```
    def __init__(self, length, width=1):
```

```
        self.length = length
```

```
        self.width = width
```

```
    def size(self, data_type=int):
```

```
        return f"The size = {data_type(self.length * self.width)}."
```

```
rectangle_1 = Rectangle(2,3)
```

```
rectangle_2 = Rectangle(3)
```

```
rectangle_3 = Rectangle(2,2)
```

```
for object in [rectangle_1, rectangle_2, rectangle_3,
```

```
Rectangle(4,4)]:
```

```
    print(object.size())
```

# `__init__` method (a dunder method)

- The `__init__` method is called automatically by Python after Python creates an object
- The object itself and the arguments are passed to the `__init__` method
- Python calls the `__init__` method for you, so the name should be exactly `__init__`
- The first thing you should do is storing the passed arguments into the objects itself, if the information is to be used outside the `__init__` method. Otherwise, the information is only available during the execution of the `__init__` method
  - `self.length = length`
  - `self.width = width`
- The `__init__` method can have a return statement, but that is only allowed to return `None`
- The `__init__` method is a so-called dunder method. Dunder means **double underscore**. Dunder methods are automatically called by Python. For example, `__add__` is automatically called when you use the plus operator. As the `__add__` dunder method can be defined for several classes, it can have a different working depending on the context. This is the so-called operator overloading
- Dunder methods are in general not directly called by you but are automatically called by Python. However, it is often possible to call them directly. The dunder method `__add__` is the method automatically called when Python processes the `+` operator, can also be called directly

```
print([1, 2, 3].__add__([4, 5]))      # On screen: [1, 2, 3, 4, 5]
```

# Class attributes

```
class Rectangle:
    total_size = 0

    def __init__(self, length, width=1):
        self.length = length
        self.width = width

        Rectangle.total_size += self.length * self.width

        print(f'{self.length} by {self.width} created, {Rectangle.total_size = }')

    def size(self, data_type=int):
        return f"The size = {data_type(self.length * self.width)}."

    def __del__(self):
        Rectangle.total_size -= self.length * self.width

        print(f'{self.length} by {self.width} deleted, {Rectangle.total_size = }')

print(Rectangle.total_size)           # On screen: 0
rectangle_1 = Rectangle(2,3)          # On screen: 2 by 3 created, Rectangle.total_size = 6

rectangle_2 = Rectangle(2)            # On screen: 2 by 1 created, Rectangle.total_size = 8
del rectangle_1                       # On screen: 2 by 3 deleted, Rectangle.total_size = 2
```

# Class attributes

- Class attributes are objects where you store information that you want to use in several objects created on the basis of the same class
- You can reference class attributes outside the class:
  - `print(Rectangle.objects_in_existence) # Onscreen: 2`



# `__del__` method

```
class Rectangle:
    objects_in_existence = 0
    def __init__(self, length, width=1):
        self.length = length
        self.width = width
        Rectangle.objects_in_existence += 1
    def size(self):
        return f"size = {self.length * self.width}"
    def info(self):
        return f'This rectangle is {self.length} by {self.width}'
    def __del__(self):
        Rectangle.objects_in_existence -= 1
rectangle_1 = Rectangle(2,3)
rectangle_2 = Rectangle(3)
```

# `__del__` method

- The `__del__` method is called automatically just before an object is deleted
- While a class definition without an `__init__` method makes no sense, a class without a `__del__` method is very common
- You can delete an object with a `del` statement, or have it automatically deleted when there is no longer any reference to this object (this is called *garbage collection*), as explained in week 1
  - In both cases Python will check whether you defined a `__del__` method and call it for you

# Referencing and defining attributes outside class and object

- You can reference and define class and object attributes from the 'outside'

```
class Rectangle:
    objects_in_existence = 0
    def __init__(self, length, width=1):
        self.length = length
        self.width = width
        Rectangle.objects_in_existence += 1
    def info_about_attribute_a(self):
        return f'{Rectangle.a=}, {self.a=}'
```

- `rectangle_1 = Rectangle(2,3)`
- **We can define attributes, and refer to attributes from outside a class/object**

```
rectangle_1 = Rectangle(2,3)
Rectangle.a = 5
rectangle_1.a = 10
print(Rectangle.a, rectangle_1.a)
#On screen: 1 5 10
```

- **Now we can define, and refer to class/object attributes from outside the class/object**

```
print(rectangle_1.info_about_attribute_a())
# On screen: Rectangle.a=5, self.a=10
```

- Though insightful, not an example to follow

# Error handling

- We saw:

```
s1 = 'abcdefghabcdefgh'
print(s1.find('i')) # Onscreen: -1
print(s1.index('i')) # Onscreen: ValueError: substring not found
```

- Both ways to tell the caller of the method that the substring is not found are fine
  - However, you could prefer the way the `find` method deals with not finding the substring over the way the `index` method deals with it (by throwing an error), or the other way around
- Python has a nice way to deal with errors, which is used a lot in the real world
  - It is not part of the exam, but it will be used in one future homework question

# Error handling

- If you want to have an index function that when not finding returns -1 instead of throwing an error, you can do the following:
  - ```
def my_index(s, needle):  
    try:  
        result = s.index(needle)
```
  - ```
    except:  
        result = -1
```
  - ```
    return result
```
  - ```
s1 = 'abcdefghabcdefgh'  
print(my_index(s1, 'e')) # Onscreen: 4  
print(my_index(s1, 'i')) # Onscreen: -1
```
- Although you can make it more complicated, this is the basis: you tell Python in the try part to run some code and tell Python in the except part, what Python should do in case of an exception (error)
- Instead of Python throwing an error and killing your program, which means that the rest of your code won't be executed, your program stays in control

# Error handling

- If you want to have a find function that when not finding throws an error, instead of returning -1, you can do the following:
  - ```
def my_find(s,needle):  
    if (result := s.find(needle)) == -1:  
        raise ValueError("substring not found")  
    return result
```
  - ```
s1 = 'abcdefghabcdefgh'  
print(my_find(s1, 'e'))    # Onscreen: 4  
print(my_find(s1, 'i'))    # Onscreen: ValueError: substring not  
found
```
- Not throwing an error can be tricky:
- ```
s1 = 'abcdefghabcdefgh'
```
- Question: What is the result of `print(s1[s1.find('i')])`